VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum, Karnataka- 590014

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

Nallanagulagari Varshita (1BF24CS188)

Under the Guidance of

Sandhya A Kulkarni

Assistant Professor, Dept of CSE

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "**OPERATING SYSTEMS – 23CS4PCOPS**" carried out by **Nallanagulagari Varshita (1BF24CS188)** who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program – 1

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→FCFS
→ SJF (pre-emptive & Non-preemptive)
→ Priority (pre-emptive & Non-pre-emptive)
→Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

FCFS

```
#include<stdio.h>
int main()
{
   int p[100],at[100],bt[100],ct[100],tat[100],wt[100],rt[100],n,g[100],c=0,a=0,b=0;
   printf("Enter no of processes : ");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
      p[i]=i;
   }
   printf("Enter Arrival time and Burst time of processes  (AT BT) \n ");
   for(int j=0;j<n;j++){
         printf("Process %d \n",j);
      scanf("%d%d",&at[j],&bt[j]);
   }
   for(int i=0;i<n-1;i++){
      for(int j=i+1;j<n;j++){
         if(at[i]>at[j]){
            int t;
            t=at[i];
            at[i]=at[j];
            at[j]=t;
```

```
            t=bt[i];
            bt[i]=bt[j];
            bt[j]=t;
            t=p[i];
            p[i]=p[j];
            p[j]=t;
          }
        }
    }
    c=0;
    for (int i=0;i<n;i++){
        if(c<at[i]){
            c=at[i];
        }
        ct[i]=c+bt[i];
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        c=ct[i];
    }
   printf(" Pid\tat\tbt\tct\ttat\twt\n");
   for(int i=0;i<n;i++){
    printf(" %d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
   }
   for(int i=0;i<n;i++){
    a=a+tat[i];
    b=b+wt[i];
   }
   printf("Avg Turn around time : %d \n Avg waiting time : %d",a/n,b/n);
    return 0;
}
```

Result:

```
Enter no of processes : 4
Enter Arrival time and Burst time of processes  (AT BT)
 Process 0
0 7
Process 1
8 3
Process 2
3 4
Process 3
5 6
 Pid    at      bt      ct      tat     wt
 0      0       7       7       7       0
 2      3       4       11      8       4
 3      5       6       17      12      6
 1      8       3       20      12      9
Avg Turn around time : 9
 Avg waiting time : 4
```

SJF (Non-preemptive):

```
#include<stdio.h>
int main()
{
   int p[100],at[100],bt[100],ct[100],tat[100],wt[100],rt[100],n,g[100],c=0,a=0,b=0,f[100]={0},
completed=0;
   printf("Enter no of processes : ");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
      p[i]=i;
   }
   printf("Enter Arrival time and Burst time of processes  (AT BT) \n ");
   for(int j=0;j<n;j++){
         printf("Process %d \n",j);
      scanf("%d%d",&at[j],&bt[j]);
   }
   while(completed<n){
      int Bursttime=1000,index=-1;
      for(int i=0;i<n;i++){
         if(f[i]==0 && at[i]<=c){
            if (bt[i]<Bursttime){
               Bursttime=bt[i];
               index=i;
            }
         }
      }
      if(index==-1){
         c++;
      }
      else{
      int i=index;
      ct[i]=c+bt[i];
      tat[i]=ct[i]-at[i];
      wt[i]=tat[i]-bt[i];
      c=ct[i];
      f[i]=1;
      completed++;
      }
   }
   printf(" Pid\tat\tbt\tct\ttat\twt\n");
  for(int i=0;i<n;i++){
   printf(" %d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
  }
```

```
   for(int i=0;i<n;i++){
    a=a+tat[i];
    b=b+wt[i];
   }
   printf("Avg Turn around time : %d \n Avg waiting time : %d",a/n,b/n);
    return 0;
}
```

Result:

```
Enter no of processes : 5
Enter Arrival time and Burst time of processes  (AT BT)
 Process 0
3 1
Process 1
1 4
Process 2
4 2
Process 3
0 6
Process 4
2 3
 Pid     at      bt      ct      tat     wt
 0       3       1       7       4       3
 1       1       4       16      15      11
 2       4       2       9       5       3
 3       0       6       6       6       0
 4       2       3       12      10      7
Avg Turn around time : 8
 Avg waiting time : 4
```

SJF(Preemptive):

```
#include<stdio.h>
int main()
{

   Int
p[100],at[100],bt[100],ct[100],tat[100],wt[100],rt[100],n,g[100],c=0,a=0,b=0,f[100]={0},compl
eted=0;
   printf("Enter no of processes : ");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
      p[i]=i;
   }
   printf("Enter Arrival time and Burst time of processes  (AT BT) \n ");
   for(int j=0;j<n;j++){
      printf("Process %d \n",j);
      scanf("%d%d",&at[j],&bt[j]);
      rt[j]=bt[j];
   }
   while(completed<n){
      int Remainingtime=1000,index=-1;
      for(int i=0;i<n;i++){
         if(rt[i]>0 && at[i]<=c){
            if (rt[i]<Remainingtime){
               Remainingtime=rt[i];
               index=i;
            }
         }
      }
      if(index==-1){
         c++;
      }
      else{
         int i=index;
         rt[i]--;
         c++;
         if (rt[i]==0){
            ct[i]=c;
            tat[i]=ct[i]-at[i];
            wt[i]=tat[i]-bt[i];
            completed++;
         }
      }
```

```
    }
    printf(" Pid\tat\tbt\tct\ttat\twt\n");
   for(int i=0;i<n;i++){
    printf(" %d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
   }
   for(int i=0;i<n;i++){
    a=a+tat[i];
    b=b+wt[i];
   }
   printf("Avg Turn around time : %d \n Avg waiting time : %d",a/n,b/n);
    return 0;
}
```

Result:

```
Enter no of processes : 5
Enter Arrival time and Burst time of processes  (AT BT)
 Process 0
2 1
Process 1
1 5
Process 2
4 1
Process 3
0 6
Process 4
2 3
 Pid    at      bt      ct      tat     wt
 0      2       1       3       1       0
 1      1       5       11      10      5
 2      4       1       5       1       0
 3      0       6       16      16      10
 4      2       3       7       5       2
Avg Turn around time : 6
 Avg waiting time : 3
Process returned 0 (0x0)   execution time : 42.083 s
Press any key to continue.
```

Priority Non-premptive:

```
#include<stdio.h>
int main()
{
   printf("Lower number ,Higher Priority\n");
   int p[100],at[100],bt[100],ct[100],tat[100],wt[100],comp[100]={0},n,c=0,a=0,b=0,pr[100],completed=0;
   printf("Enter no of processes : ");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
      p[i]=i+1;
   }
   printf("Enter Arrival time,Burst  and priority of processes  (AT BT PR) \n ");
   for(int j=0;j<n;j++){
      printf("Process %d \n",j+1);
      scanf("%d%d%d",&at[j],&bt[j],&pr[j]);
   }
   while(completed<n){
      int hp=1000,index=-1;
      for(int i=0;i<n;i++){
         if(comp[i]==0 && at[i]<=c){
            if (pr[i]<hp){
               hp=pr[i];
               index=i;
            }
         }
      }
      if(index==-1){
         c++;
      }
```

```
        else{
        int i=index;
        ct[i]=c+bt[i];
        tat[i]=ct[i]-at[i];
        wt[i]=tat[i]-bt[i];
        c=ct[i];
        comp[i]=1;
        completed++;
        }
    }
    printf(" Pid\tat\tbt\tpr\tct\ttat\twt\n");
   for(int i=0;i<n;i++){
    printf(" %d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
   }
   for(int i=0;i<n;i++){
    a=a+tat[i];
    b=b+wt[i];
   }

   printf("Avg Turn around time : %f \n Avg waiting time : %f",(float)a/n,(float)b/n);
    return 0;
}
```

Result:

```
Lower number ,Higher Priority
Enter no of processes : 5
Enter Arrival time,Burst  and priority of processes  (AT BT PR)
 Process 1
0 3 5
Process 2
2 2 3
Process 3
3 5 2
Process 4
4 4 4
Process 5
6 1 1
 Pid    at      bt      pr      ct      tat     wt
 1      0       3       5       3       3       0
 2      2       2       3       11      9       7
 3      3       5       2       8       5       0
 4      4       4       4       15      11      7
 5      6       1       1       9       3       2
Avg Turn around time : 6.200000
 Avg waiting time : 3.200000
Process returned 0 (0x0)   execution time : 12.656 s
Press any key to continue.
```

Priority premptive:

```
#include<stdio.h>
int main()
{
   printf("\nLower number ,Higher Priority\n");
   int p[100],at[100],bt[100],ct[100],tat[100],wt[100],n,c=0,a=0,b=0,pr[100],completed=0
,rt[100];
   printf("Enter no of processes : ");
   scanf("%d",&n);
   for(int i=0;i<n;i++){
      p[i]=i+1;
   }
   printf("Enter Arrival time,Burst  and priority of processes  (AT BT PR) \n ");
   for(int j=0;j<n;j++){
      printf("Process %d \n",j+1);
      scanf("%d%d%d",&at[j],&bt[j],&pr[j]);
      rt[j]=bt[j];
   }
   while(completed<n){
      int hp=1000,index=-1;
      for(int i=0;i<n;i++){
         if(rt[i]>0 && at[i]<=c){
            if (pr[i]<hp){
               hp=pr[i];
               index=i;
            }
         }
      }
      if(index==-1){
         c++;
```

```
        }
        else{
            int i=index;
            rt[i]--;
            c++;
            if (rt[i]==0){
                ct[i]=c;
                tat[i]=ct[i]-at[i];
                wt[i]=tat[i]-bt[i];
                completed++;
            }
        }
    }
    printf(" Pid\tat\tbt\tpr\tct\ttat\twt\n");
   for(int i=0;i<n;i++){
    printf(" %d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
   }
   for(int i=0;i<n;i++){
    a=a+tat[i];
    b=b+wt[i];
   }
   printf("Avg Turn around time : %f \n Avg waiting time : %f",(float)a/n,(float)b/n);
    return 0;
}
```

Result:

```
Lower number ,Higher Priority
Enter no of processes : 5
Enter Arrival time,Burst  and priority of processes  (AT BT PR)
 Process 1
0 3 5
Process 2
2 2 3
Process 3
3 5 2
Process 4
4 4 4
Process 5
6 1 1
 Pid    at      bt      pr      ct      tat     wt
 1      0       3       5       15      15      12
 2      2       2       3       10      8       6
 3      3       5       2       9       6       1
 4      4       4       4       14      10      6
 5      6       1       1       7       1       0
Avg Turn around time : 8.000000
 Avg waiting time : 5.000000
Process returned 0 (0x0)   execution time : 31.297 s
Press any key to continue.
```

Round Robin:

```
#include <stdio.h>

int main() {
   int n, tq;
   int at[100], bt[100], rt[100];
   int wt[100], tat[100], ct[100];
   int queue[100], front = 0, rear = 0;
   int visited[100] = {0};
   int time = 0, completed = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   printf("Enter Arrival time and Burst time of processes  (AT BT) \n ");
   for(int j=0;j<n;j++){
         printf("Process %d \n",j+1);
      scanf("%d%d",&at[j],&bt[j]);
      rt[j]=bt[j];
   }

   for(int i = 0; i < n; i++) {
      if(at[i] <= time && visited[i] == 0) {
         queue[rear++] = i;
         visited[i] = 1;
      }
   }

   while(completed < n) {

      if(front == rear) {
         time++;
         for(int i = 0; i < n; i++) {
            if(at[i] <= time && visited[i] == 0) {
               queue[rear++] = i;
               visited[i] = 1;
            }
         }
         continue;
      }
```

```
    int p = queue[front++];

    if(rt[p] > tq) {
        time += tq;
        rt[p] -= tq;
    } else {
        time += rt[p];
        rt[p] = 0;

        ct[p] = time;
        tat[p] = ct[p] - at[p];
        wt[p] = tat[p] - bt[p];

        completed++;
    }

    for(int i = 0; i < n; i++) {
        if(at[i] <= time && visited[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }

    if(rt[p] > 0) {
        queue[rear++] = p;
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_wt += wt[i];
    avg_tat += tat[i];
}

printf("\nAverage Waiting Time: %.2f", avg_wt / n);
printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);

return 0;
```

}

Result:

```
Enter number of processes: 5
Enter Time Quantum: 2
Enter Arrival time and Burst time of processes (AT BT)
Process 1
0 5
Process 2
1 3
Process 3
2 1
Process 4
3 2
Process 5
4 3

PID     AT      BT      CT      TAT     WT
1       0       5       13      13      8
2       1       3       12      11      8
3       2       1       5       3       2
4       3       2       9       6       4
5       4       3       14      10      7

Average Waiting Time: 5.80
Average Turnaround Time: 8.60

Process returned 0 (0x0)   execution time : 11.700 s
Press any key to continue.
```

Program – 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

Multilevel queue scheduling:

```
#include <stdio.h>
struct Process {
    int id;
    int at;
    int bt;
    int ct;
    int tat;
    int wt;
    int type;
    int completed;
};
int main() {
    int n, i;

    int completed_count = 0;
    int current_time = 0;
    float total_wt = 0, total_tat = 0;
    printf("Enter the total number of processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("\nProcess %d\n", p[i].id);
        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Enter Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Enter Process Type (1 for System, 2 for User): ");
        scanf("%d", &p[i].type);
        p[i].completed = 0;
    }
    while (completed_count < n) {
        int selected_process = -1;
```

```
        int highest_priority_type = 3;
        int earliest_arrival = 999999;
        for (i = 0; i < n; i++) {
            if (p[i].completed == 0 && p[i].at <= current_time) {
                if (p[i].type < highest_priority_type) {
                    highest_priority_type = p[i].type;
                    earliest_arrival = p[i].at;
                    selected_process = i;
                }

                else if (p[i].type == highest_priority_type && p[i].at < earliest_arrival) {
                    earliest_arrival = p[i].at;
                    selected_process = i;
                }
            }
        }
        if (selected_process != -1) {
            current_time += p[selected_process].bt;
            p[selected_process].ct = current_time;
            p[selected_process].tat = p[selected_process].ct - p[selected_process].at;
            p[selected_process].wt = p[selected_process].tat - p[selected_process].bt;
            p[selected_process].completed = 1;
            total_tat += p[selected_process].tat;
            total_wt += p[selected_process].wt;
            completed_count++;
        } else {
            current_time++;
        }
    }
    printf("\n--- Scheduling Results ---\n");
    printf("PID\tType\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
               p[i].id,
               (p[i].type == 1 ? "SYS" : "USR"),
               p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);

    return 0;
}
```

Result:

```
Enter the total number of processes: 4

Process 1
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Process Type (1 for System, 2 for User): 1

Process 2
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Process Type (1 for System, 2 for User): 1

Process 3
Enter Arrival Time: 0
Enter Burst Time: 8
Enter Process Type (1 for System, 2 for User): 2

Process 4
Enter Arrival Time: 10
Enter Burst Time: 5
Enter Process Type (1 for System, 2 for User): 1

--- Scheduling Results ---
PID     Type    AT      BT      CT      TAT     WT
1       SYS     0       4       4       4       0
2       SYS     0       3       7       7       4
3       USR     0       8       15      15      7
4       SYS     10      5       20      10      5

Average Turnaround Time: 9.00
Average Waiting Time: 4.00
PS C:\Users\BMSCECSE-L4>
```

Program – 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a)Rate- Monotonic
b)Earliest-deadline First
c)Proportional scheduling

Code:

Rate-Monotonic Scheduling:

```
#include <stdio.h>
#include <math.h>
int gcd(int a, int b) {
   while (b) {
      int t = b;
      b = a % b;
      a = t;
   }
   return a;
}
int lcm(int a, int b) {
   return (a * b) / gcd(a, b);
}

int main() {

   int n;
   printf("Enter the number of processes:");
   scanf("%d", &n);
   int C[n], P[n];
   printf("Enter the CPU burst times:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &C[i]);
   printf("Enter the time periods:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &P[i]);
   for (int i = 0; i < n - 1; i++) {
      for (int j = i + 1; j < n; j++) {
         if (P[i] > P[j]) {
            int t;
            t = P[i]; P[i] = P[j]; P[j] = t;
            t = C[i]; C[i] = C[j]; C[j] = t;
```

```
        }
    }
}
int hp = P[0];
for (int i = 1; i < n; i++)
    hp = lcm(hp, P[i]);

printf("LCM=%d\n\n", hp);
printf("Rate Monotone Scheduling:\n");
printf("PID   Burst   Period\n");
for (int i = 0; i < n; i++)
    printf("%d     %d       %d\n", i + 1, C[i], P[i]);
float u = 0;
for (int i = 0; i < n; i++)
    u += (float)C[i] / P[i];
float b = n * (pow(2, 1.0/n) - 1);
printf("\n%f <= %f =>%s\n", u, b, (u <= b) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hp);

int r[n];
for (int i = 0; i < n; i++)
    r[i] = 0;

int prev = -2;
for (int t = 0; t < hp; t++) {
    for (int i = 0; i < n; i++)
        if (t % P[i] == 0)
            r[i] = C[i];
    int f = -1;
    for (int i = 0; i < n; i++)
        if (r[i] > 0) {
            f = i;
            break;
        }
    if (f != prev) {
        if (f == -1)
            printf("%dms onwards: CPU is idle\n", t);
        else
            printf("%dms onwards: Process %d running\n", t, f + 1);
        prev = f;
    }
    if (f != -1)
        r[f]--;
}
return 0;
```

}

Result:

```
Enter the number of processes:2
Enter the CPU burst times:
20
50
Enter the time periods:
50
150
LCM=150

Rate Monotone Scheduling:
PID   Burst   Period
1     20       50
2     50       150

0.733333 <= 0.828427 =>true
Scheduling occurs for 150 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
90ms onwards: CPU is idle
100ms onwards: Process 1 running
120ms onwards: CPU is idle
```

Earliest DeadLine First:

```
#include <stdio.h>
int gcd(int a, int b) {
   while (b) {
      int t = b;
      b = a % b;
      a = t;
   }
   return a;
}


int lcm(int a, int b) {
   return (a * b) / gcd(a, b);
}

int main() {

   int n;
   printf("Enter the number of processes: ");
   scanf("%d", &n);

   int C[n], D[n], P[n], r[n];

   printf("Enter capacities:\n");
   for (int i = 0; i < n; i++) {
      scanf("%d", &C[i]);
      r[i] = 0;
   }

   printf("Enter deadlines:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &D[i]);
   printf("Enter time periods:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &P[i]);
   int hp = P[0];
   for (int i = 1; i < n; i++)
      hp = lcm(hp, P[i]);
   printf("\nLCM=%d\n\n", hp);
   printf("Earliest Deadline First Scheduling:\n");
   printf("PID  Capacity  Deadline  Period\n");
   for (int i = 0; i < n; i++)
```

```
        printf("%d     %d          %d          %d\n", i + 1, C[i], D[i], P[i]);
    float u = 0;
    for (int i = 0; i < n; i++)
        u += (float)C[i] / P[i];
    printf("\nUtilization = %.6f\n", u);
    if (u > 1.0)
        printf("Warning: Utilization > 1, schedule may be infeasible.\n");
    int time = 0, prev = -2;
    printf("\nScheduling:\n");
    while (time < hp) {
        for (int i = 0; i < n; i++) {
            if (time % P[i] == 0)
                r[i] = C[i];
        }
        int min_deadline = 1000000;
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (r[i] > 0) {
                int instances = time / P[i];
                int abs_deadline = instances * P[i] + D[i];
                if (abs_deadline < min_deadline) {
                    min_deadline = abs_deadline;
                    selected = i;
                }
            }
        }
        if (selected != prev) {
            if (selected == -1)
                printf("%dms onwards: CPU is idle\n", time);
            else
                printf("%dms onwards: Process %d running\n", time, selected + 1);
            prev = selected;
        }
        if (selected != -1)
            r[selected]--;
        time++;
    }
    printf("\nScheduling completed at %d ms\n", time);
    return 0;
}
```

Result:

```
Enter the number of processes: 3
Enter capacities:
3 2 2
Enter deadlines:
7 4 8
Enter time periods:
20 5 10

LCM=20

Earliest Deadline First Scheduling:
PID   Capacity   Deadline   Period
1     3          7          20
2     2          4          5
3     2          8          10

Utilization = 0.750000

Scheduling:
0ms onwards: Process 2 running
2ms onwards: Process 1 running
5ms onwards: Process 3 running
7ms onwards: Process 2 running
9ms onwards: CPU is idle
10ms onwards: Process 2 running
12ms onwards: Process 3 running
14ms onwards: CPU is idle
15ms onwards: Process 2 running
17ms onwards: CPU is idle
```

Program – 4

Write a C program to simulate
a) producer-consumer problem using semaphores
b) concept of Dining Philosophers problem.

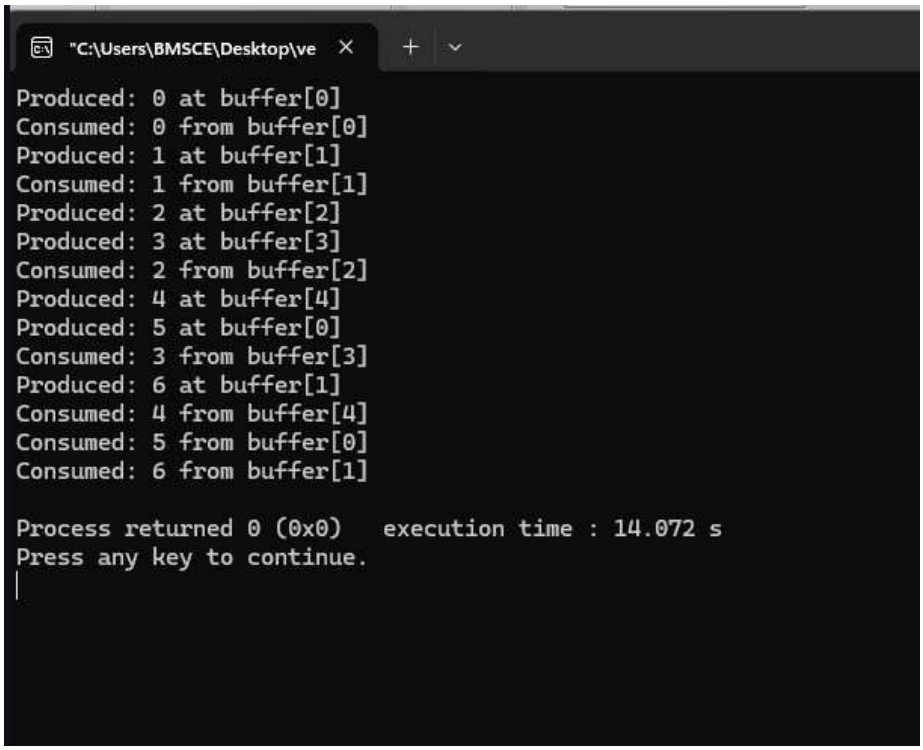
Code:

Producer consumer problem:

```
#include <stdio.h>

int in = 0, out = 0;
int mutex = 1, Full = 0, Empty = 10;
int a[10];

void wait(int *s)
{
   while (*s <= 0)
      ;
   (*s)--;
}

void signal(int *s)
{
   (*s)++;
}

int main()
{
   int p, item,c=0;;


   while (1)
   {
      printf("\n1: Produce\n2: Consume\n3: Exit\nEnter choice: ");
      scanf("%d", &p);
      if (p == 1)
      {
         wait(&Empty);
         wait(&mutex);
         printf("Enter number to produce: ");
         scanf("%d", &item);
         if(c==10){
```

```
            printf("Over FLow");
        }
        a[in] = item;
        in = (in + 1) % 10;
        c++;
        printf("Produced: %d\n", item);
        signal(&mutex);
        signal(&Full);
    }
    else if (p == 2)
    {
        wait(&Full);
        wait(&mutex);
        if(c==0){
            printf("Under FLow");
        }
        item = a[out];
        out = (out + 1) % 10;
        printf("Consumed: %d\n", item);
        c--;
        signal(&mutex);
        signal(&Empty);
    }
    else if (p == 3)
    {
        break;
    }
    else
    {
        printf("Invalid choice\n");
    }
  }
  return 0;
}
```

Result:


"C:\Users\BMSCE\Desktop\ve
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Consumed: 4 from buffer[4]
Consumed: 5 from buffer[0]
Consumed: 6 from buffer[1]
Process returned 0 (0x0) execution time : 14.072 s
Press any key to continue.

Dining Philosopher:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void *philosopher(void *num)
{
   int id = *(int *)num;
   int left = id;
   int right = (id + 1) % N;
   while (1)
   {
      printf("Philosopher %d is thinking\n", id);
      sleep(1);
      if (id % 2 == 0)
      {
         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d\n", id, left);
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d\n", id, right);
      }
      else
      {
         pthread_mutex_lock(&forks[right]);
         printf("Philosopher %d picked up right fork %d\n", id, right);

         pthread_mutex_lock(&forks[left]);
         printf("Philosopher %d picked up left fork %d\n", id, left);
      }
      printf("Philosopher %d is eating\n", id);
      sleep(2);
      pthread_mutex_unlock(&forks[left]);
      pthread_mutex_unlock(&forks[right]);
      printf("Philosopher %d put down forks %d and %d\n",
            id, left, right);
   }
   return NULL;
}
```

```
int main()
{
   int i, ids[N];

   for (i = 0; i < N; i++)
   {
      pthread_mutex_init(&forks[i], NULL);
      ids[i] = i;
   }
   for (i = 0; i < N; i++)
   {
      pthread_create(&philosophers[i], NULL,
                  philosopher, &ids[i]);
   }
   for (i = 0; i < N; i++)
   {
      pthread_join(philosophers[i], NULL);
   }

   for (i = 0; i < N; i++)
   {
      pthread_mutex_destroy(&forks[i]);
   }

   return 0;
}
```

Result:

```
Philosopher 1 is thinking
Philosopher 0 is thinking
Philosopher 3 is thinking
Philosopher 2 is thinking
Philosopher 4 is thinking
Philosopher 1 picked up right fork 2
Philosopher 1 picked up left fork 1
Philosopher 1 is eating
Philosopher 0 picked up left fork 0
Philosopher 3 picked up right fork 4
Philosopher 3 picked up left fork 3
Philosopher 3 is eating
Philosopher 1 put down forks 1 and 2
Philosopher 1 is thinking
Philosopher 2 picked up left fork 2
Philosopher 3 put down forks 3 and 4
Philosopher 3 is thinking
Philosopher 0 picked up right fork 1
Philosopher 0 is eating
Philosopher 2 picked up right fork 3
Philosopher 2 is eating
Philosopher 4 picked up left fork 4
Philosopher 0 put down forks 0 and 1
Philosopher 0 is thinking
Philosopher 4 picked up right fork 0
Philosopher 1 picked up right fork 2
Philosopher 1 picked up left fork 1
```

Program – 5

Write a C program to simulate

a) Bankers algorithm for the purpose of deadlock avoidance

b) Deadlock Detection

Code:

Banker's Algorithm:

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   printf("Enter number of resources: ");
   scanf("%d", &m);

   int alloc[n][m], max[n][m], need[n][m], avail[m];

   printf("Enter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("Enter Max Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &max[i][j]);
      }
   }

   printf("Enter Available Resources:\n");
   for(i = 0; i < m; i++)
```

```
{
   scanf("%d", &avail[i]);
}

for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
   {
      need[i][j] = max[i][j] - alloc[i][j];
   }
}

int finish[n], safeSeq[n], work[m];
int count = 0;

for(i = 0; i < m; i++)
   work[i] = avail[i];

for(i = 0; i < n; i++)
   finish[i] = 0;

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         for(j = 0; j < m; j++)
         {
            if(need[i][j] > work[j])
               break;
         }

         if(j == m)
         {
            for(k = 0; k < m; k++)
               work[k] += alloc[i][k];

            safeSeq[count++] = i;
            finish[i] = 1;
            found = 1;
         }
      }
```

```
        }

        if(found == 0)
        {
            printf("\nSystem is not in a safe state.\n");
            return 0;
        }
    }

    printf("\nSystem is in a safe state.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safeSeq[i]);
        if(i != n - 1)
            printf(" -> ");
    }

    printf("\n");

    return 0;
}
```

Result:

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
```

Deadlock Detection:

```
#include <stdio.h>

int main()
{
   int n, m, i, j, k;
   printf("Enter number of resources: ");
   scanf("%d", &m);

   printf("Enter number of processes: ");
   scanf("%d", &n);

   int alloc[n][m], req[n][m], avail[m];
   int finish[n], work[m], safeSeq[n];
   int count = 0;

   printf("Enter Allocation Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }

   printf("Enter Request Matrix:\n");
   for(i = 0; i < n; i++)
   {
      for(j = 0; j < m; j++)
      {
         scanf("%d", &req[i][j]);
      }
   }

   printf("Enter Available Resources:\n");
   for(i = 0; i < m; i++)
   {
      scanf("%d", &avail[i]);
   }

   for(i = 0; i < n; i++)
      finish[i] = 0;
```

```
for(i = 0; i < m; i++)
   work[i] = avail[i];

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         int flag = 1;

         for(j = 0; j < m; j++)
         {
            if(req[i][j] > work[j])
            {
               flag = 0;
               break;
            }
         }

         if(flag)
         {
            for(k = 0; k < m; k++)
            {
               work[k] += alloc[i][k];
            }

            finish[i] = 1;
            safeSeq[count++] = i;
            found = 1;
         }
      }
   }

   if(found == 0)
      break;
}

if(count == n)
{
   printf("\nSystem is in safe state\n");
   printf("Safe Sequence: ");
```

```
        for(i = 0; i < n; i++)
        {
            printf("P%d", safeSeq[i]);

            if(i != n - 1)
                printf(" -> ");
        }
    }
    else
    {
        printf("\nDeadlock detected!\n");
        printf("Processes in deadlock are: ");

        for(i = 0; i < n; i++)
        {
            if(finish[i] == 0)
                printf("P%d ", i);
        }
    }

    return 0;
}
```

Result:

```
Enter number of resources: 3
Enter number of processes: 5
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter Available Resources:
0 0 0

System is in safe state
Safe Sequence: P0 -> P2 -> P3 -> P4 -> P1
```

Program – 6

Write a C program to simulate the following contiguous memory allocation techniques.
a) Worst-fit
b) Best-fit
c) First-fit

Code:

Continuous Memory Allocation:

```
#include <stdio.h>

void firstFit(int bs[], int b, int ps[], int p)
{
   int a[p];
   int u[b];

   for(int i = 0; i < p; i++)
      a[i] = -1;

   for(int i = 0; i < b; i++)
      u[i] = 0;

   for(int i = 0; i < p; i++)
   {
      for(int j = 0; j < b; j++)
      {
         if(!u[j] && bs[j] >= ps[i])
         {
            a[i] = j;
            u[j] = 1;
            break;
         }
      }
   }

   printf("\nFirst Fit\n");
   printf("Process\tSize\tBlock\n");

   for(int i = 0; i < p; i++)
   {
      printf("P%d\t%d\t", i + 1, ps[i]);
```

```
        if(a[i] != -1)
            printf("%d\n", a[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int bs[], int b, int ps[], int p)
{
    int a[p];
    int u[b];

    for(int i = 0; i < p; i++)
        a[i] = -1;

    for(int i = 0; i < b; i++)
        u[i] = 0;

    for(int i = 0; i < p; i++)
    {
        int bestIdx = -1;

        for(int j = 0; j < b; j++)
        {
            if(!u[j] && bs[j] >= ps[i])
            {
                if(bestIdx == -1 || bs[j] < bs[bestIdx])
                    bestIdx = j;
            }
        }

        if(bestIdx != -1)
        {
            a[i] = bestIdx;
            u[bestIdx] = 1;
        }
    }

    printf("\nBest Fit\n");
    printf("Process\tSize\tBlock\n");
    for(int i = 0; i < p; i++)
    {
        printf("P%d\t%d\t", i + 1, ps[i]);
```

```
        if(a[i] != -1)
            printf("%d\n", a[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int bs[], int b, int ps[], int p)
{
    int a[p];
    int u[b];

    for(int i = 0; i < p; i++)
        a[i] = -1;

    for(int i = 0; i < b; i++)
        u[i] = 0;
    for(int i = 0; i < p; i++)
    {
        int worstIdx = -1;

        for(int j = 0; j < b; j++)
        {
            if(!u[j] && bs[j] >= ps[i])
            {
                if(worstIdx == -1 || bs[j] > bs[worstIdx])
                    worstIdx = j;
            }
        }
        if(worstIdx != -1)
        {
            a[i] = worstIdx;
            u[worstIdx] = 1;
        }
    }
    printf("\nWorst Fit\n");
    printf("Process\tSize\tBlock\n");
    for(int i = 0; i < p; i++)
    {
        printf("P%d\t%d\t", i + 1, ps[i]);

        if(a[i] != -1)
            printf("%d\n", a[i] + 1);
        else
            printf("Not Allocated\n");
```

```
    }
}
int main()
{
int blocks, processes;
    printf("Enter number of blocks: ");
    scanf("%d", &blocks);
    int blockSize[blocks];
    printf("Enter block sizes:\n");
    for(int i = 0; i < blocks; i++)
        scanf("%d", &blockSize[i]);
    printf("Enter number of processes: ");
    scanf("%d", &processes);

    int processSize[processes];
    printf("Enter process sizes:\n");
    for(int i = 0; i < processes; i++)
        scanf("%d", &processSize[i]);
    firstFit(blockSize, blocks, processSize, processes);
    bestFit(blockSize, blocks, processSize, processes);
    worstFit(blockSize, blocks, processSize, processes);
    return 0;
}
```

Result:

```
Enter number of blocks: 5
Enter block sizes:
400 700 200 300 600
Enter number of processes: 4
Enter process sizes:
212 512 312 526

First Fit
Process Size     Block
P1       212     1
P2       512     2
P3       312     5
P4       526     Not Allocated

Best Fit
Process Size     Block
P1       212     4
P2       512     5
P3       312     1
P4       526     2

Worst Fit
Process Size     Block
P1       212     2
P2       512     5
P3       312     1
P4       526     Not Allocated
```

Program – 7

Write a C program to simulate page replacement algorithms.
a) FIFO
b)LRU
c)Optimal

Code:

Page Replacement Algorithms:

```
#include <stdio.h>
void simulate(int p[], int n, int c, int mode)
{
   int f[20], faults = 0, next_fifo = 0;
   int last_used[20];

   for(int i = 0; i < c; i++)
   {
      f[i] = -1;
      last_used[i] = 0;
   }
   printf("\n%s Page Replacement:\n",
         mode == 0 ? "FIFO" :
         mode == 1 ? "Optimal" : "LRU");
   for(int i = 0; i < n; i++)
   {
      int found = -1, empty = -1;

      for(int j = 0; j < c; j++)
      {
         if(f[j] == p[i])
            found = j;
```

```
    if(f[j] == -1 && empty == -1)
        empty = j;
}

if(found != -1)
{
    if(mode == 2)
        last_used[found] = i;
}
else
{
    faults++;
    int rep = empty;
    if(rep == -1)
    {
        if(mode == 0)      /* FIFO */
        {
            rep = next_fifo;
            next_fifo = (next_fifo + 1) % c;
        }
        else if(mode == 1) /* Optimal */
        {
            int farthest = -1;
            rep = 0;
            for(int j = 0; j < c; j++)
            {
                int next_use = 9999;
                for(int k = i + 1; k < n; k++)
                {
                    if(f[j] == p[k])
                    {
```

```
                    next_use = k;
                    break;
                }
            }
            if(next_use > farthest)
            {
                farthest = next_use;
                rep = j;
            }
        }
    }
    else            /* LRU */
    {
        rep = 0;
        for(int j = 1; j < c; j++)
        {
            if(last_used[j] < last_used[rep])
                rep = j;
        }
    }
  }
  f[rep] = p[i];
  if(mode == 2)
    last_used[rep] = i;
}
printf("Page %d -> ", p[i]);
for(int j = 0; j < c; j++)
{
  if(f[j] == -1)
    printf("- ");
  else
```

```
            printf("%d ", f[j]);
        }
        if(found == -1)
            printf("(Fault)");
        printf("\n");
    }
    printf("Total Page Faults: %d\n", faults);
}
int main()
{
    int p[50], n, frames;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &p[i]);
    printf("Enter number of frames: ");
    scanf("%d", &frames);
    for(int m = 0; m < 3; m++)
        simulate(p, n, frames, m);
    return 0;
}
```

Result:

```
Enter number of pages: 12
Enter reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

FIFO Page Replacement:
Page 7 -> 7 - - (Fault)
Page 0 -> 7 0 - (Fault)
Page 1 -> 7 0 1 (Fault)
Page 2 -> 2 0 1 (Fault)
Page 0 -> 2 0 1
Page 3 -> 2 3 1 (Fault)
Page 0 -> 2 3 0 (Fault)
Page 4 -> 4 3 0 (Fault)
Page 2 -> 4 2 0 (Fault)
Page 3 -> 4 2 3 (Fault)
Page 0 -> 0 2 3 (Fault)
Page 3 -> 0 2 3
Total Page Faults: 10
```

```
Optimal Page Replacement:
Page 7 -> 7 - - (Fault)
Page 0 -> 7 0 - (Fault)
Page 1 -> 7 0 1 (Fault)
Page 2 -> 2 0 1 (Fault)
Page 0 -> 2 0 1
Page 3 -> 2 0 3 (Fault)
Page 0 -> 2 0 3
Page 4 -> 2 4 3 (Fault)
Page 2 -> 2 4 3
Page 3 -> 2 4 3
Page 0 -> 0 4 3 (Fault)
Page 3 -> 0 4 3
Total Page Faults: 7

LRU Page Replacement:
Page 7 -> 7 - - (Fault)
Page 0 -> 7 0 - (Fault)
Page 1 -> 7 0 1 (Fault)
Page 2 -> 2 0 1 (Fault)
Page 0 -> 2 0 1
Page 3 -> 2 0 3 (Fault)
Page 0 -> 2 0 3
Page 4 -> 4 0 3 (Fault)
Page 2 -> 4 0 2 (Fault)
Page 3 -> 4 3 2 (Fault)
Page 0 -> 0 3 2 (Fault)
Page 3 -> 0 3 2
Total Page Faults: 9
```